

神经网络编程框架实验报告

一、实验主要内容

本次大作业的目标是实现一个简单的神经网络编程框架。框架需要覆盖神经网络训练流程中的主要组成部分，包括数据读取与批量加载、张量运算、自动微分、模块化网络层、参数初始化、优化器、学习率调度器、模型训练测试流程以及模型参数的保存和读取。

结合课程 PDF 和项目代码结构，本实验主要完成以下内容：

1. 实现 `Dataset` 和 `DataLoader`，支持从表格数据和图像数据中读取样本，并按 mini-batch 形式提供给训练过程。数据集目标包括 Iris、MNIST 和 Fashion-MNIST。
2. 实现张量计算和自动微分的核心机制，包括计算图节点 `Value`、张量 `Tensor`、计算操作 `Op` / `TensorOp`、拓扑排序和反向传播。
3. 实现常用张量算子，例如逐元素加减乘除、矩阵乘法、转置、变形、广播、求和、对数、指数和 ReLU 等。
4. 实现模块化神经网络组件，包括 `Module`、`Parameter`、`Linear`、`Flatten`、`ReLU`、`Sequential`、`Residual` 以及常用损失函数。
5. 实现 Xavier 和 He 参数初始化方法。
6. 实现优化器和学习率规划器，包括 SGD、Momentum、Adam、weight decay，以及 StepDecay、LinearWarmUp、CosineDecayWithWarmRestarts 等学习率调度策略。
7. 使用该简易框架完成 ResidualMLP 实验，并实现训练、测试、模型保存和模型读取。测试阶段应避免构建无用梯度。

二、实验计划与步骤

本项目代码骨架较大，但当前并不是一个完整可运行的框架。因此实验按“先打通最小训练闭环，再逐步补充功能”的顺序推进。

第一步，修复项目导入入口。当前 `needle` 顶层导入会经过 `needle.nn`，而 `needle.nn.__init__` 中导入了缺失的 `nn_sequence.py`，会导致 `import needle` 失败。因此需要先调整 `python/needle/nn/__init__.py`，只导入本实验需要的基础模块。同时修复 `TensorOp.__call__` 中中文说明未写成注释的问题。

第二步，补齐自动微分核心流程。实现 `python/needle/autograd.py` 中的拓扑排序函数 `find_topo_sort` 和反向传播函数 `compute_gradient_of_variables`。这是后续所有 `loss.backward()` 和优化器更新参数的基础。

第三步，实现 ResidualMLP 需要的基础张量算子。优先补齐 `ops_mathematic.py` 中的 `PowerScalar`、`EwiseDiv`、`DivScalar`、`Transpose`、`Reshape`、`BroadcastTo`、`Summation`、`MatMul`、`Negate`、`Log`、`Exp` 和 `ReLU`。这些算子足以支持全连接网络、损失函数和自动求导测试。

第四步，实现参数初始化和基础网络模块。在 `init_initializers.py` 中实现 Xavier/He 初始化；在 `nn_basic.py` 中实现 `Sequential`、损失函数、`Residual` 等模块，使网络结构可以通过模块组合的方式定义。

第五步，实现优化器和学习率调度器。在 `optim.py` 中补齐 SGD、Momentum、Adam 和 weight decay；随后增加 Scheduler 相关类，实现学习率随训练轮数变化。

第六步，实现数据读取与批量加载。完成 `DataLoader`，实现 MNIST/Fashion-MNIST 文件解析，并新增 Iris 数据读取支持。先用 Iris 或小规模 MNIST 子集测试训练闭环，再扩大到完整数据集。

第七步，实现 ResidualMLP 实验脚本。新增模型定义、训练、验证、测试、模型保存和读取逻辑。训练过程中记录 loss 和 accuracy，实验报告中展示训练曲线和最终测试结果。

第八步，补充测试和实验分析。对自动微分使用数值微分做梯度检查；对 DataLoader、初始化、优化器和 ResidualMLP 训练过程分别进行小规模验证。

三、自动微分核心实现

本阶段完成的是实验计划中的第二步，即实现自动微分的核心流程。修改位置为 `python/needle/autograd.py` 中的两个函数：

- `find_topo_sort(node_list)`
- `compute_gradient_of_variables(output_tensor, out_grad)`

1. 拓扑排序

计算图中的每个 `value / Tensor` 节点只保存自己的输入节点 `inputs`，并不保存输出节点。因此拓扑排序需要从输出节点出发，沿着 `inputs` 反向遍历整张计算图。

实现时采用深度优先搜索的后序遍历：

1. 从目标输出节点开始 DFS。
2. 对当前节点，先递归访问它的所有输入节点。
3. 输入节点都访问完后，再把当前节点加入 `topo_order`。
4. 使用 `visited` 集合避免重复访问共享子图中的节点。

这样得到的拓扑序满足：一个节点的输入一定排在该节点之前。也就是说，前向依赖关系是从前到后排列的。反向传播时只需要将这个拓扑序反转，就可以从输出节点逐步回传到叶子节点。

2. 反向传播

反向传播函数 `compute_gradient_of_variables` 的核心思想是维护一个字典：

```
1 | node_to_output_grads_list: Dict[Tensor, List[Tensor]]
```

该字典记录每个节点从所有输出边收到的局部伴随值，也就是 partial adjoint。由于同一个中间节点可能被多个后续节点使用，因此它可能收到多份梯度贡献。反向传播时需要先把这些梯度贡献求和，得到该节点最终的 `node.grad`。

具体流程如下：

1. 先将输出节点的初始梯度设为 `out_grad`。
2. 对输出节点做拓扑排序，并反转拓扑序。
3. 按反向拓扑序遍历每个节点。
4. 将当前节点收到的所有梯度贡献用 `sum_node_list` 相加，保存到 `node.grad`。
5. 如果当前节点是叶子节点，则没有前驱操作，不继续传播。
6. 如果当前节点不是叶子节点，则调用 `node.op.gradient_as_tuple(node.grad, node)`，计算该操作对每个输入节点的梯度贡献。
7. 将每个输入节点收到的新梯度追加到 `node_to_output_grads_list` 中。

这一实现与课程 PDF 中给出的自动求导伪代码一致：先按反向拓扑序求当前节点的总伴随值，再通过当前节点对应的局部梯度规则，把梯度继续传回输入节点。

3. 当前实现的意义

完成这一步后，框架具备了自动微分的基本调度能力。后续只要每个 `TensorOp` 正确实现自己的 `gradient` 方法，整个计算图就可以自动完成链式求导。例如，矩阵乘法、广播、求和、ReLU 和交叉熵损失等算子的梯度规则补齐后，就可以支持 `loss.backward()`，并让优化器读取每个参数的 `param.grad` 来更新网络参数。

本阶段只实现自动微分的图遍历和梯度传播框架，并未实现具体算子的梯度公式。具体 `TensorOp` 的前向计算和局部梯度将在后续步骤中完成。

四、基础张量算子实现

本阶段完成的是实验计划中的第三步，即补齐 `ops_mathematic.py` 和 `ops_logarithmic.py` 中的基础张量算子。该步骤的目标是让前一步实现的自动微分框架真正可以对神经网络中的常用运算进行前向计算和反向求导。

本次实现的算子包括：

- `PowerScalar`
- `EwiseDiv`
- `DivScalar`
- `Transpose`
- `Reshape`
- `BroadcastTo`
- `Summation`
- `MatMul`
- `Negate`
- `Log`
- `Exp`
- `ReLU`
- `Tanh`
- `LogSoftmax`
- `LogSumExp`

此外，还修正了 `EwisePow` 的实现。原始代码中 `EwisePow.gradient` 判断输入类型的逻辑不正确，因为计算图中 `node.inputs` 保存的是 `Tensor` 节点，而不是底层的 `NDArray` 数据。同时，`needle` 后端中底层 `NDArray.__pow__` 路径并不稳定，因此对 `EwisePow.compute` 和 `PowerScalar.compute` 分别处理了 NumPy 后端和 `needle` 后端。

1. 基础逐元素算子

逐元素算子包括幂、除法、取负、对数、指数、ReLU 和 Tanh。这些算子的前向计算直接调用后端提供的数组计算接口，反向传播根据基本求导公式实现。

主要梯度公式如下：

```
1 d(a ** c) / da = c * a ** (c - 1)
2 d(a / b) / da = 1 / b
3 d(a / b) / db = -a / b^2
4 d(log a) / da = 1 / a
5 d(exp a) / da = exp(a)
6 d(tanh a) / da = 1 - tanh(a)^2
```

对于 ReLU，前向计算为 $\max(a, 0)$ ，反向传播时使用输入是否大于 0 的 mask：

```
1 grad = out_grad * (a > 0)
```

这里的 mask 通过输入节点的 `cached_data` 构造为不需要梯度的 `Tensor`，再参与反向传播计算。

2. 形状变换算子

`Transpose` 和 `Reshape` 这类算子只改变张量的视图或形状，不改变元素的数值关系。

`Reshape` 的反向传播比较直接：前向从原形状变为目标形状，反向时只需要把上游梯度 reshape 回输入张量的原始形状。

`Transpose` 的反向传播是再次交换相同的轴。因为转置操作本身是一个置换，逆操作仍然可以用相同轴对的转置表示。对于未指定轴的情况，默认交换最后两个维度，与矩阵转置一致。

实现时还考虑了后端差异：`needle` 后端使用 `array_api.permute`，NumPy 后端使用 `np.transpose`。

3. 广播和求和

`BroadcastTo` 和 `Summation` 是一对在反向传播中经常互相出现的操作。

对于 `Summation`，前向会把某些维度消去。反向传播时，需要先把上游梯度 reshape 成带有被消去维度的形状，再 broadcast 回输入张量原始形状。例如，对形状 (m, n) 的张量按第 1 维求和后，输出形状为 $(m,)$ ，反向时需要先变成 $(m, 1)$ ，再广播为 (m, n) 。

对于 `BroadcastTo`，前向会把某些长度为 1 的维度扩展到更大的尺寸，或者在前面补出新维度。反向传播时，需要把这些被广播出来的维度求和压回原来的形状。为此实现了辅助函数 `_sum_to_shape`，专门处理“将梯度规约回指定 shape”的逻辑。

这一部分是后续实现 bias 加法、BatchNorm、LayerNorm 和损失函数时必须依赖的基础。

4. 矩阵乘法

`MatMul` 的前向计算使用矩阵乘法。对于普通二维矩阵：

```
1 C = A @ B
```

其梯度公式为：

```
1 dL/dA = dL/dC @ B^T
2 dL/dB = A^T @ dL/dC
```

实现中还兼容了 batched matmul。当输入存在 batch 维度或发生广播时，反向传播得到的梯度形状可能大于原输入形状，因此同样使用 `_sum_to_shape` 把梯度规约回原始输入 shape。

这使得该算子既能支持普通全连接层中的二维矩阵乘法，也可以支持更高维张量中的批量矩阵乘法。

5. LogSoftmax 和 LogSumExp

`LogSoftmax` 和 `LogSumExp` 是分类损失函数中非常重要的数值稳定算子。为了避免直接计算 `exp` 时溢出，`LogSumExp` 使用稳定形式：

```
1 logsumexp(Z) = max(Z) + log(sum(exp(Z - max(Z))))
```

`LogSoftmax` 的前向计算可以写成：

```
1 logsoftmax(Z) = Z - logsumexp(Z)
```

其中 `logsumexp` 默认沿最后一个维度计算，并广播回原始形状。

`LogSumExp` 的梯度本质上是 softmax 权重：

```
1 d logsumexp(Z) / dZ = exp(Z - logsumexp(Z))
```

`LogSoftmax` 的梯度为：

```
1 dL/dZ = out_grad - softmax(Z) * sum(out_grad)
```

实现时还补充了多轴 reduction 的 shape 处理，包括 `axes=None`、负轴、单轴和多轴。由于底层 `NDArray.sum/max` 对多轴规约支持有限，代码在 op 层按轴从大到小逐个规约，并通过 `_keepdims_shape` 和 `_remove_axes_shape` 恢复需要的形状。

6. 检查结果

完成实现后，对两个算子文件进行了语法检查，并确认 `ops_mathematic.py` 和 `ops_logarithmic.py` 中已经没有剩余的 `raise NotImplementedError()`。

随后进行了若干小规模功能测试：

1. 在 `needle` 后端下测试矩阵乘法、`logsumexp`、`logsoftmax` 的组合前向与反向传播，梯度能够正常计算。
2. 在 NumPy 后端下测试同样的组合算子，前向和反向传播结果正常。
3. 测试 batched matmul，输出形状和两个输入的梯度形状都符合预期。
4. 测试 `logsumexp` 的 `axes=None`、`axes=0`、`axes=1`、`axes=(0, 1)` 和 `axes=-1`，输出形状和数值正常。
5. 对 `power`、`log`、`exp`、`tanh` 的组合函数与 NumPy 解析梯度进行比较，最大梯度误差为 0。

这些检查说明基础张量算子已经可以接入自动微分框架，为后续实现神经网络模块、损失函数和优化器提供了必要的运算基础。

五、初始化方法与基础神经网络模块实现

本阶段完成的是实验计划中的第四步，即补齐 `init_initializers.py` 中的参数初始化函数，以及 `nn_basic.py` 中的基础神经网络模块。该步骤的目标是让框架能够通过模块组合的方式定义神经网络，并为后续 ResidualMLP 实验提供必要的层、激活函数、损失函数和正则化组件。

本次实现的初始化函数包括：

- `xavier_uniform`
- `xavier_normal`
- `kaiming_uniform`
- `kaiming_normal`

本次实现或补充的神经网络模块包括：

- `Sequential`
- `Sigmoid`
- `Softmax`
- `SoftmaxLoss`
- `CrossEntropyLoss`
- `BinaryCrossEntropyLoss`
- `MSELoss`
- `BatchNorm1d`
- `LayerNorm1d`
- `Dropout`
- `Residual`
- `CrossEntropyLoss`
- `BinaryCrossEntropyLoss`

其中 `CrossEntropyLoss` 和 `BinaryCrossEntropyLoss` 保留了原始代码中的拼写，另外新增了正确拼写的别名 `CrossEntropyLoss` 和 `BinaryCrossEntropyLoss`，方便后续实验脚本使用。

1. 参数初始化

参数初始化函数位于 `python/needle/init/init_initializers.py`。本实验实现了 Xavier 和 Kaiming/He 两类初始化方法。

Xavier 初始化主要适用于 Sigmoid、Tanh 等激活函数，目标是让前向传播和反向传播中的方差尽量稳定。其均匀分布形式为：

```
1 | W ~ U(-a, a)
2 | a = gain * sqrt(6 / (fan_in + fan_out))
```

正态分布形式为：

```
1 | W ~ N(0, std^2)
2 | std = gain * sqrt(2 / (fan_in + fan_out))
```

Kaiming/He 初始化主要适用于 ReLU 激活函数。其均匀分布形式为：

```
1 | W ~ U(-a, a)
2 | a = sqrt(6 / fan_in)
```

正态分布形式为：

```
1 | W ~ N(0, std^2)
2 | std = sqrt(2 / fan_in)
```

实现时，如果用户没有显式传入 `shape`，默认使用 `(fan_in, fan_out)`，可以直接用于全连接层的权重初始化。同时，为了避免参数和输入张量落在不同后端设备上，初始化函数在没有显式传入 `device` 时使用 `default_device()`。

2. 顺序模型与残差连接

`Sequential` 用于把多个模块顺序组合起来。其前向传播逻辑是依次调用内部保存的每个子模块：

```
1 | for module in self.modules:
2 |     x = module(x)
3 | return x
```

这样可以用较简洁的方式定义多层神经网络，例如 `Linear -> ReLU -> Linear`。

`Residual` 用于实现残差连接。其前向传播为：

```
1 | return self.fn(x) + x
```

即先将输入传入子模块 `fn`，再把子模块输出与原输入相加。该模块是后续实现 `ResidualMLP` 的核心组件。

3. 激活函数与 Softmax

`ReLU` 已经通过 `ops.relu` 接入张量算子。本阶段新增了 `Sigmoid` 和 `Softmax`。

`Sigmoid` 使用指数函数实现：

```
1 | sigmoid(x) = 1 / (1 + exp(-x))
```

由于当前框架中的 `Tensor.__rsub__` 对 `1 - tensor` 这类表达式支持不完善，实现时尽量使用不会触发该问题的等价写法。

`Softmax` 使用前一阶段实现的稳定 `logsoftmax`：

```
1 | softmax(x) = exp(logsoftmax(x))
```

这样可以复用 `LogSoftmax` 中的数值稳定处理。

4. 损失函数

`softmaxLoss` 用于多分类任务，输入为 logits 和类别标签。实现时没有直接计算 softmax，而是使用稳定的 log-sum-exp 形式：

```
1 | loss = logsumexp(logits) - logits[y]
```

在代码中，先将类别标签转为 one-hot，再计算每个样本的交叉熵，最后对 batch 求平均。

`CrossEntropyLoss` 用于标签已经是 one-hot 或概率分布的情况。它先计算：

```
1 | log_probs = logsoftmax(logits)
```

然后计算：

```
1 | loss = -sum(y * log_probs) / batch_size
```

`BinaryCrossEntropyLoss` 用于二分类概率输入，公式为：

```
1 | loss = -[y * log(p) + (1 - y) * log(1 - p)]
```

实现时加入很小的 `eps`，避免出现 `log(0)`。同时为了绕开当前 `Tensor.__rsub__` 的限制，代码中使用 `-y + 1` 和 `-p + 1` 表示 `1 - y` 和 `1 - p`。

`MSELoss` 用于回归任务，计算预测值和目标值之间的均方误差：

```
1 | loss = mean((input - target)^2)
```

5. BatchNorm1d

`BatchNorm1d` 对二维输入 `(batch_size, dim)` 的每个特征维度进行归一化。训练阶段先计算 batch 均值和方差：

```
1 | mean = sum(x, axis=0) / batch_size
2 | var = sum((x - mean)^2, axis=0) / batch_size
```

然后进行归一化和仿射变换：

```
1 | y = (x - mean) / sqrt(var + eps) * weight + bias
```

其中 `weight` 和 `bias` 是可训练参数，形状为 `(dim,)`。

训练阶段还会使用 momentum 更新 `running_mean` 和 `running_var`：

```
1 | running = (1 - momentum) * running + momentum * batch_stat
```

测试阶段则不再使用当前 batch 的统计量，而是使用训练过程中累计的 running mean 和 running var。

实现时需要显式 reshape 和 broadcast 均值、方差、权重和偏置，因为当前框架的逐元素加减乘除不会自动完成所有广播。

6. LayerNorm1d

`LayerNorm1d` 对每个样本的最后一维进行归一化。与 `BatchNorm` 不同，`LayerNorm` 不依赖 batch 统计量，而是在每个样本内部计算均值和方差：

```
1 mean = mean(x, last_dim)
2 var = mean((x - mean)^2, last_dim)
```

随后同样进行归一化和仿射变换。为了保留维度，代码中实现了 `_sum_keepdims` 和 `_mean_keepdims` 辅助函数，先沿指定轴求和，再 reshape 成 `keepdims` 的形式，便于后续 broadcast。

7. Dropout

`Dropout` 在训练阶段随机将部分元素置零，并对剩余元素进行缩放：

```
1 y = x * mask / (1 - p)
```

其中 `mask` 以概率 `1 - p` 取 1，以概率 `p` 取 0。这样可以保证训练阶段输出的期望与输入一致。

在 eval 模式下，`Dropout` 直接返回原输入，不再随机置零。

8. 检查结果

完成实现后，对 `init_initializers.py` 和 `nn_basic.py` 进行了语法检查，并确认两个文件中已经没有剩余的 `raise NotImplementedError()`。

随后进行了若干小规模测试：

1. 四个初始化函数都能生成形状正确的张量。
2. `Sequential`、`Linear`、`ReLU`、`Sigmoid`、`Residual` 和 `SoftmaxLoss` 可以组成小型网络，并完成前向和反向传播。
3. `BatchNorm1d`、`LayerNorm1d` 和 `Dropout` 的前向输出形状正确，并可以参与反向传播。
4. `BatchNorm1d` 在 eval 模式下可以使用 running mean 和 running var。
5. NumPy 后端下，`Sequential + MSELoss` 的前向和反向传播正常。
6. `CrossEntropyLoss` 和 `BinaryCrossEntropyLoss` 的前向和反向传播正常。

这些检查说明基础模块已经能够接入前面实现的张量算子和自动微分系统，为后续实现优化器、数据加载和 `ResidualMLP` 实验提供了模块化网络构建能力。

六、优化器实现

本阶段完成的是实验计划中的第五步，即补齐 `python/needle/optim.py` 中的优化器实现。优化器负责读取自动微分阶段写入到参数上的梯度，并根据指定更新规则修改模型参数。

本次实现的内容包括：

- `SGD.step`

- `SGD.clip_grad_norm`
- `Adam.step`

同时，为了避免优化器内部状态被意外接入计算图，本阶段还对优化器状态和参数更新过程做了显式 `detach` 处理。

1. Optimizer 基类

项目中的优化器都继承自 `Optimizer` 基类。基类保存参数列表：

```
1 | self.params = params
```

并提供 `reset_grad` 方法：

```
1 | for p in self.params:  
2 |     p.grad = None
```

一次标准训练迭代通常为：

```
1 | loss.backward()  
2 | optimizer.step()  
3 | optimizer.reset_grad()
```

其中 `loss.backward()` 负责计算梯度，`optimizer.step()` 负责更新参数，`optimizer.reset_grad()` 负责清空旧梯度，避免下一轮反向传播时梯度残留。

2. SGD 参数更新

`SGD.step` 遍历所有参数，并跳过 `grad is None` 的参数。普通 SGD 的更新公式为：

```
1 | param = param - lr * grad
```

如果设置了 weight decay，则使用 L2 正则化形式修正梯度：

```
1 | grad = grad + weight_decay * param
```

如果设置了 momentum，则使用速度缓存 `u`：

```
1 | u = momentum * u + grad  
2 | param = param - lr * u
```

代码中使用参数对象本身作为字典 key：

```
1 | self.u[param]
```

用于保存每个参数独立的动量状态。

这里采用的是常见深度学习框架中的 SGD momentum 写法，即 $u = \text{momentum} * u + \text{grad}$ 。另一种写法 $u = \text{momentum} * u + (1 - \text{momentum}) * \text{grad}$ 更接近指数滑动平均，会改变学习率的实际尺度。本实验采用前者，使其行为更接近常见的 SGD momentum。

3. 梯度裁剪

`SGD.clip_grad_norm` 用于在参数更新前限制梯度整体大小，避免梯度爆炸。

首先计算所有参数梯度的全局 L2 范数：

```
1 total_norm = sqrt(sum(grad^2))
```

如果该范数超过给定阈值 `max_norm`，就对所有参数梯度乘以同一个缩放系数：

```
1 scale = max_norm / (total_norm + 1e-6)
2 grad = grad * scale
```

这样可以保持所有参数梯度的相对方向不变，只缩小整体长度。函数返回裁剪前的 `total_norm`，便于后续训练时记录日志或分析训练稳定性。

4. Adam 参数更新

Adam 使用一阶矩和二阶矩对梯度进行自适应缩放。每次调用 `step` 时，先将时间步 `t` 加一：

```
1 self.t += 1
```

然后对每个参数维护两个状态：

```
1 u: 梯度一阶矩
2 v: 梯度二阶矩
```

其更新公式为：

```
1 u = beta1 * u + (1 - beta1) * grad
2 v = beta2 * v + (1 - beta2) * grad^2
```

由于初始时 `u` 和 `v` 为 0，Adam 需要进行 bias correction：

```
1 u_hat = u / (1 - beta1^t)
2 v_hat = v / (1 - beta2^t)
```

最终参数更新为：

```
1 param = param - lr * u_hat / (sqrt(v_hat) + eps)
```

如果设置了 weight decay，则与 SGD 一样，先对梯度加入 L2 正则项：

```
1 grad = grad + weight_decay * param
```

5. detach 处理

优化器的状态不应该成为神经网络计算图的一部分。否则随着训练迭代增加，优化器状态可能保留旧计算图，导致不必要的内存占用和错误的梯度依赖。

因此本阶段对以下数据显式调用了 `detach()`：

1. 进入优化器更新逻辑的 `param.grad`。
2. SGD 的 momentum 缓存 `self.u[param]`。
3. 梯度裁剪后写回的 `param.grad`。
4. Adam 的一阶矩 `self.u[param]`。
5. Adam 的二阶矩 `self.v[param]`。
6. Adam 中用于参数更新的 `update`。
7. 写回参数的 `param.data`。

例如，SGD momentum 缓存更新后会执行：

```
1 | self.u[param] = self.u[param].detach()
```

Adam 中的一阶矩和二阶矩更新后也会执行：

```
1 | self.u[param] = self.u[param].detach()
2 | self.v[param] = self.v[param].detach()
```

这样可以保证优化器只保存数值状态，而不保存训练计算图。

6. 检查结果

完成实现后，对 `optim.py` 进行了语法检查。随后构造小型网络，分别测试了普通 SGD、带 momentum 和 weight decay 的 SGD，以及 Adam。

测试流程为：

1. 构造一个小型 `Sequential` 网络。
2. 前向计算 `MSELoss`。
3. 调用 `loss.backward()` 计算梯度。
4. 对 SGD 调用 `clip_grad_norm`。
5. 调用 `optimizer.step()` 更新参数。
6. 调用 `optimizer.reset_grad()` 清空梯度。

检查结果表明：

1. SGD、带 momentum/weight decay 的 SGD 和 Adam 都能完成一次参数更新。
2. 更新后参数数值确实发生变化。
3. `reset_grad` 后所有参数梯度被清空。
4. 默认 needle 后端和 NumPy 后端的小模型测试均通过。

这些检查说明优化器已经可以和前面实现的自动微分、张量算子、神经网络模块配合使用，后续可以接入完整训练循环。

七、ResidualMLP 实验实现

本阶段完成的是实验计划中的第六步和第七步，即完成 ResidualMLP 实验所需的数据处理、模型定义、训练脚本、模型保存与加载，以及初步训练测试。

本阶段新增的文件包括：

- `data/prepare_data.py`
- `apps/residual_mlp.py`
- `apps/train_residual_mlp.py`

1. 数据集下载与本地组织

实验使用的数据集包括 Iris、MNIST 和 Fashion-MNIST。为了避免每次训练时重复下载数据，本实验采用“先下载到本地，再预处理为统一格式”的方式管理数据。

原始数据保存在：

```
1 | data/raw/
```

预处理后的数据保存在：

```
1 | data/processed/
```

最终目录结构如下：

```
1 | data/  
2 |   raw/  
3 |     iris/  
4 |     mnist/  
5 |     fashion-mnist/  
6 |   processed/  
7 |     iris/  
8 |     mnist/  
9 |     fashion-mnist/
```

Iris 原始数据为文本格式，每行包含 4 个连续特征和 1 个类别标签。MNIST 和 Fashion-MNIST 原始数据为 IDX gzip 格式，包含图像文件和标签文件。

2. 数据预处理脚本

数据预处理脚本为 `data/prepare_data.py`。该脚本只负责从本地 `data/raw` 中读取数据，不负责联网下载。

运行命令为：

```
1 | python data/prepare_data.py --dataset all --image-shape flat
```

其中 `--dataset` 可以选择：

```
1 all
2 iris
3 mnist
4 fashion-mnist
```

`--image-shape flat` 表示将 MNIST 和 Fashion-MNIST 图像保存为展平后的 $(N, 784)$ 。如果使用 `--image-shape image`，则保存为 $(N, 28, 28)$ 。

脚本对 MNIST/Fashion-MNIST 的处理流程如下：

1. 使用 `gzip` 读取 `.gz` 文件。
2. 使用 `struct.unpack` 解析 IDX 文件头。
3. 检查 magic number，确保图像文件和标签文件格式正确。
4. 图像数据从 `uint8` 转为 `float32`。
5. 将像素值除以 255，归一化到 $[0, 1]$ 。
6. 标签转为 `int8`。
7. 保存为 `.npz` 文件。

Iris 的处理流程如下：

1. 读取 `iris.data` 或 `bezdekIris.data`。
2. 前 4 列作为 `float32` 特征。
3. 类别标签映射为整数：

```
1 Iris-setosa -> 0
2 Iris-versicolor -> 1
3 Iris-virginica -> 2
```

4. 使用分层划分，按类别保持比例划分训练集和测试集。
5. 默认测试集比例为 0.2，即 120 条训练样本、30 条测试样本。
6. 保存为 `.npz` 文件。

预处理后生成的数据文件为：

```
1 data/processed/iris/train.npz
2 data/processed/iris/test.npz
3 data/processed/mnist/train.npz
4 data/processed/mnist/test.npz
5 data/processed/fashion-mnist/train.npz
6 data/processed/fashion-mnist/test.npz
```

实际生成的数据形状如下：

```

1  Iris:
2      train X=(120, 4), y=(120,)
3      test  X=(30, 4),  y=(30,)
4
5  MNIST:
6      train X=(60000, 784), y=(60000,)
7      test  X=(10000, 784), y=(10000,)
8
9  Fashion-MNIST:
10     train X=(60000, 784), y=(60000,)
11     test  X=(10000, 784), y=(10000,)

```

检查结果表明，MNIST 和 Fashion-MNIST 的图像数据范围为 `[0, 1]`，标签类别为 `0-9`；Iris 标签类别为 `0, 1, 2`。

3. Dataset 与 DataLoader 实现

在完成数据预处理脚本后，本阶段进一步补全了框架内部的数据读取模块，使训练脚本不再依赖手写的 `numpy batch` 迭代器，而是统一通过 `Dataset` 和 `DataLoader` 提供数据。

修改的主要文件包括：

- `python/needle/data/data_basic.py`
- `python/needle/data/data_transforms.py`
- `python/needle/data/datasets/mnist_dataset.py`
- `python/needle/data/datasets/iris_dataset.py`
- `python/needle/data/datasets/__init__.py`

`DataLoader` 的核心职责是把任意 `Dataset` 按 mini-batch 形式迭代出来，并将 `numpy` 数据转换为框架中的 `Tensor`。实现时维护一个 `ordering` 数组表示当前 epoch 的样本顺序：

```

1  indices = np.random.permutation(len(dataset)) if shuffle else np.arange(len(dataset))

```

然后使用 `np.array_split` 按 `batch_size` 划分为多个 batch。每次调用 `__iter__` 时都会重新生成 `ordering`，因此当 `shuffle=True` 时，每个 epoch 都会重新打乱训练样本。

`DataLoader.__next__` 的处理流程为：

1. 如果所有 batch 已经迭代完成，则抛出 `StopIteration`。
2. 取出当前 batch 对应的样本下标。
3. 如果数据集没有 transform，则优先使用 `dataset[batch_indices]` 做批量索引。
4. 如果数据集带有 transform，则逐样本调用 `dataset[int(i)]`，确保随机翻转、随机裁剪等增强对每个样本独立生效。
5. 将样本用 `np.stack` 合并为 batch。
6. 将 `numpy` 数组递归转换为 `needle.Tensor`。

这样设计后，训练循环只需要写：

```

1 for x_tensor, y_tensor in dataloader:
2     logits = model(x_tensor)
3     loss = loss_fn(logits, y_tensor)

```

不再需要在训练脚本中手动维护样本下标、batch 切分和 Tensor 构造。

MNIST 数据集通过 `parse_mnist` 解析 gzip 压缩的 IDX 文件。图像文件头使用 `struct.unpack(">IIII", ...)` 读取 magic number、样本数、行数和列数；标签文件头使用 `struct.unpack(">II", ...)` 读取 magic number 和标签数。实现中检查 magic number，避免图像文件和标签文件传错。图像数据读取后 reshape 为 `(N, 784)`，再转为 `float32` 并除以 255 归一化到 `[0, 1]`；标签转为 `int8`。

Fashion-MNIST 与 MNIST 使用相同的 IDX 文件格式，因此实现为：

```

1 class FashionMNISTDataset(MNISTDataset):
2     pass

```

也就是复用 MNIST 的解析逻辑。

Iris 数据集新增为 `IrisDataset`。该类既支持读取预处理后的 `.npz` 文件，也支持读取原始 `iris.data` 文本文件。读取 `.npz` 时直接加载 `x` 和 `y`；读取原始文本时，将 4 个特征转为 `float32`，并将三个类别映射为整数标签：

```

1 Iris-setosa -> 0
2 Iris-versicolor -> 1
3 Iris-virginica -> 2

```

此外，还补全了 `RandomFlipHorizontal` 和 `RandomCrop`。前者按概率对 HWC 图像沿宽度方向翻转，后者先对图像做零填充，再根据随机偏移裁剪回原始大小。虽然当前 ResidualMLP 主要使用展平图像，但这两个 transform 使数据模块对后续图像实验也可用。

本阶段对数据模块做了小规模检查：

```

1 Iris processed: len=120, batch=(16, 4), labels=(16,)
2 Iris raw: len=150, full batch=(150, 4)
3 MNIST raw: len=60000, batch=(32, 784), range=[0, 1]
4 Fashion-MNIST raw: len=60000, batch=(32, 784)
5 NDArrayDataset + DataLoader: batch 切分正常
6 Transform smoke test: 随机翻转/裁剪 shape 正常

```

这些检查说明 `DataLoader` 可以统一服务表格数据、图像数据和已有的 `NDArrayDataset`。

4. ResidualMLP 模型结构

ResidualMLP 模型定义在 `apps/residual_mlp.py`。该模型用于表格数据或展平后的图像数据。输入为：

```

1 (batch_size, input_dim)

```

输出为：

```

1 (batch_size, num_classes)

```

整体结构为：

```
1 input
2 -> Linear(input_dim, hidden_dim)
3 -> ReLU
4 -> ResidualMLPBlock * num_blocks
5 -> Linear(hidden_dim, num_classes)
```

每个残差块 `ResidualMLPBlock` 的结构为：

```
1 x
2 -> Residual(
3     Linear(hidden_dim, hidden_dim)
4     ReLU
5     Linear(hidden_dim, hidden_dim)
6 )
7 -> ReLU
```

也就是：

```
1 block(x) = ReLU(x + Linear(ReLU(Linear(x))))
```

这种结构先通过输入层将原始特征映射到统一的 hidden 维度，然后在 hidden 空间中堆叠残差块。残差连接要求输入输出维度一致，因此残差块内部全部使用 `hidden_dim -> hidden_dim` 的线性层。

模型构造函数支持以下参数：

```
1 ResidualMLP(
2     input_dim,
3     hidden_dim,
4     num_classes,
5     num_blocks=2,
6     device=None,
7     dtype="float32",
8 )
```

其中：

- `input_dim` 为输入维度，Iris 为 4，MNIST/Fashion-MNIST 为 784。
- `hidden_dim` 为隐藏层宽度。
- `num_classes` 为分类类别数。
- `num_blocks` 为残差块数量。

在 `forward` 中，如果输入维度大于 2，例如 `(N, 1, 28, 28)`，模型会自动展平成 `(N, 784)`，从而兼容图像格式输入和已经展平的输入。

5. 训练脚本实现

训练脚本为 `apps/train_residual_mlp.py`。在补全 `DataLoader` 后，训练脚本从原先的手写 numpy batch 迭代器改为统一使用框架数据模块。

脚本新增了两个数据入口函数：

```
1 build_datasets(name)
2 build_data loaders(name, batch_size)
```

`build_datasets` 根据数据集名称构造对应的 Dataset：

1. `iris` 使用 `IrisDataset` 读取 `data/processed/iris/train.npz` 和 `test.npz`。
2. `mnist` 使用 `MNISTDataset` 读取 `data/raw/mnist` 下的 gzip IDX 文件。
3. `fashion-mnist` 使用 `FashionMNISTDataset` 读取 `data/raw/fashion-mnist` 下的 gzip IDX 文件。

`build_data loaders` 在此基础上构造训练和测试 DataLoader：

```
1 DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
2 DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

训练集开启 shuffle，测试集不打乱顺序。

训练脚本支持以下功能：

1. 从本地 Dataset 构造 DataLoader。
2. 根据数据集自动设置输入维度和类别数。
3. 构造 `ResidualMLP`。
4. 选择优化器：`SGD` 或 `Adam`。
5. 使用 `SoftmaxLoss` 训练分类模型。
6. 每个 epoch 后计算训练集和测试集的 loss/accuracy。
7. 支持梯度裁剪。
8. 支持保存模型参数。
9. 支持加载模型参数。
10. 支持 eval-only 模式，只评估模型。

训练循环的核心流程为：

```
1 for x_tensor, y_tensor in dataloader:
2     logits = model(x_tensor)
3     loss = loss_fn(logits, y_tensor)
4
5     if train:
6         loss.backward()
7         optimizer.step()
8         optimizer.reset_grad()
```

其中 `x_tensor` 和 `y_tensor` 已经由 `DataLoader` 转换为 `needle.Tensor`，因此训练脚本不再需要手动执行：

```
1 x_tensor = nd1.Tensor(xb, dtype="float32")
2 y_tensor = nd1.Tensor(yb, dtype="float32")
```

每个 batch 的准确率统计通过 `y_tensor.numpy().astype("int32")` 取回标签，并与 `logits.numpy().argmax(axis=1)` 比较。

单次参数更新仍然遵循：

```
1 logits = model(x_tensor)
2 loss = loss_fn(logits, y_tensor)
3 loss.backward()
4 optimizer.step()
5 optimizer.reset_grad()
```

测试阶段调用：

```
1 model.eval()
```

并且只进行前向计算、loss 计算和 accuracy 统计，不调用 `backward()`，从而满足“测试阶段不计算梯度”的要求。

模型参数保存使用 `np.savez_compressed`，将 `model.parameters()` 中的每个参数按顺序保存为：

```
1 param_0
2 param_1
3 ...
```

加载模型时，按相同顺序读取参数，并写回每个 `param.data`。

6. 运行命令

Iris 短测试命令：

```
1 PYTHONPATH=./python python apps/train_residual_mlp.py \
2   --dataset iris \
3   --epochs 3 \
4   --batch-size 16 \
5   --hidden-dim 16 \
6   --num-blocks 1 \
7   --optimizer adam \
8   --lr 0.01 \
9   --save-path checkpoints/test_residual_mlp_iris.npz
```

加载保存模型并只做测试：

```
1 PYTHONPATH=./python python apps/train_residual_mlp.py \  
2 --dataset iris \  
3 --batch-size 16 \  
4 --hidden-dim 16 \  
5 --num-blocks 1 \  
6 --load-path checkpoints/test_residual_mlp_iris.npz \  
7 --eval-only
```

MNIST smoke test 命令:

```
1 PYTHONPATH=./python python apps/train_residual_mlp.py \  
2 --dataset mnist \  
3 --epochs 1 \  
4 --batch-size 256 \  
5 --hidden-dim 32 \  
6 --num-blocks 1 \  
7 --optimizer adam \  
8 --lr 0.001 \  
9 --save-path checkpoints/test_residual_mlp_mnist.npz
```

7. 实验结果

Iris 数据集上, 使用 3 个 epoch 的短测试结果如下:

```
1 epoch=001 train_loss=3.251963 train_acc=0.2083 test_loss=2.018009 test_acc=0.3333  
2 epoch=002 train_loss=1.497287 train_acc=0.3167 test_loss=1.037383 test_acc=0.4667  
3 epoch=003 train_loss=0.905957 train_acc=0.5917 test_loss=0.641000 test_acc=0.6667  
4 saved checkpoint: checkpoints/test_residual_mlp_iris.npz
```

可以看到, 在很短的训练轮数内, 训练 loss 和测试 loss 都明显下降, 测试准确率从 0.3333 提升到 0.6667, 说明模型、损失函数、优化器和训练循环能够正常协同工作。

随后使用保存的 checkpoint 进行 eval-only 测试, 结果为:

```
1 loaded checkpoint: checkpoints/test_residual_mlp_iris.npz  
2 eval test_loss=0.641000 test_acc=0.6667
```

该结果与训练结束时的测试结果一致, 说明模型参数保存和读取流程正常。

MNIST 数据集上, 使用 1 个 epoch 的 smoke test 结果如下:

```
1 epoch=001 train_loss=0.628530 train_acc=0.8137 test_loss=0.270481 test_acc=0.9231  
2 saved checkpoint: checkpoints/test_residual_mlp_mnist.npz
```

该实验只用于验证大规模数据集路径、batch 训练、模型前向/反向、优化器更新和 checkpoint 保存是否正常。结果显示, 在 MNIST 上训练 1 个 epoch 后, 测试准确率达到 0.9231, 说明当前 ResidualMLP 实验脚本已经可以在实际数据集上完成完整训练流程。

将训练脚本切换到 DataLoader 后, 又进行了以下检查:

```

1  Iris eval-only:
2  eval test_loss=13.444977 test_acc=0.3333
3
4  MNIST DataLoader:
5  (64, 784) (64,)
6
7  Fashion-MNIST DataLoader:
8  (64, 784) (64,)
9
10 Iris 训练 1 epoch:
11 epoch=001 train_loss=6.267969 train_acc=0.3333 test_loss=3.523802 test_acc=0.3333
12 saved checkpoint: checkpoints/test_data_loader_residual_mlp_iris.npz
13
14 加载 checkpoint eval:
15 eval test_loss=3.523802 test_acc=0.3333

```

其中第一次保存 checkpoint 时，由于当前命令运行环境是只读沙箱，保存文件时报出只读文件系统错误。授权写入后重跑同一命令通过，说明该问题来自运行环境限制，不是训练脚本逻辑错误。

8. 小结

本阶段完成了从本地数据读取、预处理、Dataset/DataLoader 批量加载、模型定义、训练、测试、保存和加载的完整实验闭环。至此，框架已经可以使用自定义的 Tensor、自动微分、神经网络模块、优化器和数据加载模块完成 ResidualMLP 分类实验。

八、学习率调度器实现与接入

本阶段完成学习率调度器 Scheduler 的实现，并将其接入 ResidualMLP 训练脚本。学习率调度器的作用是在训练过程中动态修改优化器中的 `optimizer.lr`，从而控制每个 epoch 或 step 使用的学习率。它本身不直接更新模型参数，参数更新仍由 `SGD` 或 `Adam` 完成。

1. Scheduler 基类设计

学习率调度器直接写入 `python/needle/optim.py`。本阶段新增了一个基类：

```

1  class LRScheduler:
2      def __init__(self, optimizer):
3          self.optimizer = optimizer
4          self.base_lr = optimizer.lr
5          self.last_epoch = -1
6
7      def step(self):
8          self.last_epoch += 1
9          lr = self.get_lr()
10         self.optimizer.lr = lr
11         return lr

```

其中：

1. `optimizer` 是需要被调度的优化器对象，例如 `SGD` 或 `Adam`。
2. `base_lr` 保存优化器初始学习率，作为后续计算学习率的基准。

3. `last_epoch` 记录当前调度步数，初始值设为 `-1`，这样第一次调用 `step()` 后进入第 0 个调度点。
4. `step()` 调用子类的 `get_lr()` 得到新学习率，并写回 `optimizer.lr`。

这样设计后，所有具体调度策略只需要实现 `get_lr()`，训练脚本只需要统一调用：

```
1 | lr = scheduler.step()
```

2. StepDecay

`StepDecay` 实现阶梯学习率衰减。其参数包括：

```
1 | StepDecay(optimizer, step_size, gamma=0.1)
```

学习率公式为：

```
1 | lr = base_lr * gamma ^ (last_epoch // step_size)
```

也就是说，每经过 `step_size` 个调度步，学习率乘以一次 `gamma`。例如 `base_lr=0.1`、`step_size=2`、`gamma=0.5` 时，学习率序列为：

```
1 | 0.1, 0.1, 0.05, 0.05, 0.025, ...
```

该策略适合在训练中后期逐步降低学习率，使参数更新幅度变小。

3. LinearWarmUp

`LinearWarmUp` 实现线性预热。其参数包括：

```
1 | LinearWarmUp(optimizer, warmup_steps, start_lr=0.0)
```

在前 `warmup_steps` 个调度步内，学习率从 `start_lr` 线性增加到 `base_lr`：

```
1 | lr = start_lr + (base_lr - start_lr) * (last_epoch + 1) / warmup_steps
```

当 warmup 结束后，学习率保持为 `base_lr`。例如 `base_lr=0.1`、`warmup_steps=4`、`start_lr=0.0` 时，学习率序列为：

```
1 | 0.025, 0.05, 0.075, 0.1, 0.1, ...
```

使用 warmup 的意义是避免训练初期学习率过大导致 loss 剧烈震荡，尤其适合较深网络或较大 batch 的训练。

4. CosineDecayWithWarmRestarts

`CosineDecayWithWarmRestarts` 实现带重启的余弦退火。其参数包括：

```

1 CosineDecayWithWarmRestarts(
2     optimizer,
3     first_cycle_steps,
4     min_lr=0.0,
5     cycle_mult=1.0,
6 )

```

在每个周期内部，学习率按余弦曲线从 `base_lr` 衰减到接近 `min_lr`：

```

1 lr = min_lr + (base_lr - min_lr) * (1 + cos(pi * cycle_step / cycle_steps)) / 2

```

当一个周期结束后，学习率重新回到 `base_lr`，进入下一个周期。`cycle_mult` 用于控制后续周期长度是否增长。若 `cycle_mult=1.0`，每个周期长度相同；若大于 1，则后续周期逐渐变长。

例如 `base_lr=0.1`、`first_cycle_steps=4`、`min_lr=0.01` 时，学习率序列为：

```

1 0.1, 0.08682, 0.055, 0.02318, 0.1, ...

```

这种策略可以在训练过程中周期性增大学习率，有助于模型跳出较差的局部区域。

5. 训练脚本接入

训练脚本 `apps/train_residual_mlp.py` 新增了 `build_scheduler(args, optimizer)` 函数，根据命令行参数构造对应的 scheduler：

```

1 def build_scheduler(args, optimizer):
2     if args.scheduler == "none":
3         return None
4     if args.scheduler == "step":
5         return ndl.optim.StepDecay(...)
6     if args.scheduler == "warmup":
7         return ndl.optim.LinearWarmUp(...)
8     if args.scheduler == "cosine":
9         return ndl.optim.CosineDecayWithWarmRestarts(...)

```

新增的命令行参数包括：

```

1 --scheduler {none,step,warmup,cosine}
2 --step-size
3 --gamma
4 --warmup-steps
5 --warmup-start-lr
6 --cosine-first-cycle-steps
7 --cosine-min-lr
8 --cosine-cycle-mult

```

其中 `--scheduler` 默认值为 `none`，因此如果不显式指定 scheduler，训练脚本行为与原来保持一致。

训练循环中，在每个 epoch 开始时调用 scheduler：

```

1 for epoch in range(1, args.epochs + 1):
2     lr = scheduler.step() if scheduler is not None else optimizer.lr
3     train_loss, train_acc = run_epoch(...)
4     test_loss, test_acc = run_epoch(...)

```

这样保证当前 epoch 的训练实际使用的就是日志中打印出的 `lr`。如果把 scheduler 放在 epoch 末尾调用，那么 warmup 的第一轮训练仍会使用原始 `base_lr`，不符合预热的直觉。因此本实验采用 epoch 开始调度的方式。

日志输出中也加入当前学习率：

```

1 epoch=001 train_loss=... train_acc=... test_loss=... test_acc=... lr=...

```

6. 使用示例

不使用 scheduler:

```

1 python apps/train_residual_mlp.py \
2     --dataset iris \
3     --epochs 2 \
4     --scheduler none

```

使用 StepDecay:

```

1 python apps/train_residual_mlp.py \
2     --dataset iris \
3     --epochs 2 \
4     --scheduler step \
5     --step-size 1 \
6     --gamma 0.5

```

使用 LinearWarmUp:

```

1 python apps/train_residual_mlp.py \
2     --dataset iris \
3     --epochs 2 \
4     --scheduler warmup \
5     --warmup-steps 2 \
6     --warmup-start-lr 0.0

```

使用 CosineDecayWithWarmRestarts:

```

1 python apps/train_residual_mlp.py \
2     --dataset iris \
3     --epochs 2 \
4     --scheduler cosine \
5     --cosine-first-cycle-steps 2 \
6     --cosine-min-lr 0.0001

```

7. 检查结果

首先单独检查三个 scheduler 产生的学习率序列：

```
1 StepDecay:
2 [0.1, 0.1, 0.05, 0.05, 0.025]
3
4 LinearWarmUp:
5 [0.025, 0.05, 0.075, 0.1, 0.1, 0.1]
6
7 CosineDecayWithWarmRestarts:
8 [0.1, 0.08682, 0.055, 0.02318, 0.1, 0.08682, 0.055, 0.02318, 0.1]
```

随后验证训练脚本的默认行为和三种 scheduler 参数均能正常运行。默认 `eval-only` 结果为：

```
1 eval test_loss=13.444977 test_acc=0.3333
```

StepDecay 短训练结果为：

```
1 epoch=001 train_loss=6.839057 train_acc=0.3333 test_loss=5.789763 test_acc=0.3333
  lr=0.001
2 epoch=002 train_loss=5.455952 train_acc=0.3333 test_loss=5.014381 test_acc=0.3333
  lr=0.0005
3 saved checkpoint: checkpoints/test_step_scheduler.npz
```

LinearWarmUp 短训练结果为：

```
1 epoch=001 train_loss=7.175205 train_acc=0.3333 test_loss=6.637601 test_acc=0.3333
  lr=0.0005
2 epoch=002 train_loss=5.962381 train_acc=0.3333 test_loss=5.040403 test_acc=0.3333
  lr=0.001
3 saved checkpoint: checkpoints/test_warmup_scheduler.npz
```

CosineDecayWithWarmRestarts 短训练结果为：

```
1 epoch=001 train_loss=6.839057 train_acc=0.3333 test_loss=5.789763 test_acc=0.3333
  lr=0.001
2 epoch=002 train_loss=5.423942 train_acc=0.3333 test_loss=4.939537 test_acc=0.3333
  lr=0.00055
3 saved checkpoint: checkpoints/test_cosine_scheduler.npz
```

这些检查说明 scheduler 能正确修改优化器学习率，并且已经成功接入 ResidualMLP 训练流程。

九、ResidualMLP 系统实验

本阶段对 ResidualMLP 进行了完整系统实验。实验目标是固定模型结构，只改变数据集、优化器和学习率调度策略，比较不同训练配置对最终效果的影响。

1. 实验设置

本次系统实验固定模型结构为：

```
1 ResidualMLP
2 hidden_dim = 128
3 num_blocks = 2
4 seed = 42
5 epochs = 10
6 batch_size = 256
```

实验数据集包括：

```
1 Iris
2 MNIST
3 Fashion-MNIST
```

优化器包括：

```
1 SGD
2 Adam
```

学习率调度策略包括：

```
1 none
2 step
3 warmup
4 cosine
```

因此总实验数为：

```
1 3 datasets * 2 optimizers * 4 schedulers = 24
```

2. 训练脚本修改

为了支持系统实验，对 `apps/train_residual_mlp.py` 做了进一步扩展。

首先，训练脚本新增了实验目录管理逻辑。每组实验会自动在 `checkpoints` 下创建独立子目录，目录名包含模型名、数据集、优化器、scheduler、模型大小和学习率，例如：

```
1 checkpoints/ResidualMLP__mnist__adam__cosine_c5_min0p0001_mult1__hd128_blk2__lr0p001/
```

每个实验目录中保存：

```
1 config.json
2 metrics.csv
3 summary.json
4 curves.png
5 model.npz
```

其中 `metrics.csv` 记录每个 epoch 的训练损失、训练准确率、测试损失、测试准确率和学习率；`summary.json` 记录最终结果和最佳测试准确率；`curves.png` 保存 loss/accuracy 曲线；`model.npz` 保存模型参数。

其次，训练脚本新增了 `tqdm` 进度条。每个训练和测试 epoch 都显示 batch 级进度，同时在进度条中显示当前累计 loss 和 accuracy。为了便于在无交互终端运行，脚本还提供：

```
1 | --no-progress
```

用于关闭进度条。

最后，在 `apps/train_residual_mlp.sh` 中整理了 24 组实验命令。脚本按数据集、优化器和 scheduler 三层循环运行，每组实验开始前先输出当前配置，再调用训练脚本。实验命令说明保留在 `apps/residual_mlp_experiments.txt` 中。

3. 实验配置表

编号	数据集	优化器	Scheduler	学习率	Scheduler 参数
1	Iris	SGD	none	0.01	-
2	Iris	SGD	step	0.01	step_size=5, gamma=0.5
3	Iris	SGD	warmup	0.01	warmup_steps=3, start_lr=0.0
4	Iris	SGD	cosine	0.01	first_cycle_steps=5, min_lr=0.0001
5	Iris	Adam	none	0.001	-
6	Iris	Adam	step	0.001	step_size=5, gamma=0.5
7	Iris	Adam	warmup	0.001	warmup_steps=3, start_lr=0.0
8	Iris	Adam	cosine	0.001	first_cycle_steps=5, min_lr=0.0001
9	MNIST	SGD	none	0.01	-
10	MNIST	SGD	step	0.01	step_size=5, gamma=0.5
11	MNIST	SGD	warmup	0.01	warmup_steps=3, start_lr=0.0
12	MNIST	SGD	cosine	0.01	first_cycle_steps=5, min_lr=0.0001
13	MNIST	Adam	none	0.001	-
14	MNIST	Adam	step	0.001	step_size=5, gamma=0.5
15	MNIST	Adam	warmup	0.001	warmup_steps=3, start_lr=0.0
16	MNIST	Adam	cosine	0.001	first_cycle_steps=5, min_lr=0.0001
17	Fashion-MNIST	SGD	none	0.01	-
18	Fashion-MNIST	SGD	step	0.01	step_size=5, gamma=0.5
19	Fashion-MNIST	SGD	warmup	0.01	warmup_steps=3, start_lr=0.0

编号	数据集	优化器	Scheduler	学习率	Scheduler 参数
20	Fashion-MNIST	SGD	cosine	0.01	first_cycle_steps=5, min_lr=0.0001
21	Fashion-MNIST	Adam	none	0.001	-
22	Fashion-MNIST	Adam	step	0.001	step_size=5, gamma=0.5
23	Fashion-MNIST	Adam	warmup	0.001	warmup_steps=3, start_lr=0.0
24	Fashion-MNIST	Adam	cosine	0.001	first_cycle_steps=5, min_lr=0.0001

4. 实验结果

本次实验共生成 24 个实验目录。检查结果表明，每个目录均包含 `metrics.csv`、`summary.json`、`curves.png` 和 `model.npz`，没有出现 NaN/Inf，也没有出现画图错误。

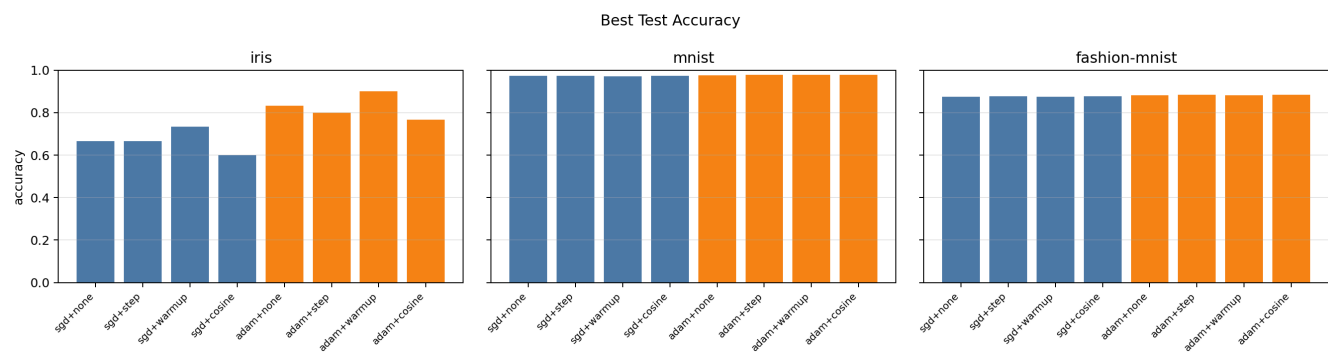
24 组实验结果如下：

数据集	优化器	Scheduler	Final Test Acc	Best Test Acc	Best Epoch	Final Test Loss
Fashion-MNIST	Adam	cosine	0.8787	0.8847	9	0.3292
Fashion-MNIST	Adam	none	0.8711	0.8809	7	0.3605
Fashion-MNIST	Adam	step	0.8767	0.8836	8	0.3422
Fashion-MNIST	Adam	warmup	0.8781	0.8811	8	0.3413
Fashion-MNIST	SGD	cosine	0.8760	0.8762	9	0.3440
Fashion-MNIST	SGD	none	0.8729	0.8749	9	0.3542
Fashion-MNIST	SGD	step	0.8779	0.8779	10	0.3414
Fashion-MNIST	SGD	warmup	0.8694	0.8755	9	0.3550
Iris	Adam	cosine	0.6000	0.7667	6	1.6603
Iris	Adam	none	0.6000	0.8333	8	1.4667
Iris	Adam	step	0.6333	0.8000	9	0.8634
Iris	Adam	warmup	0.6667	0.9000	6	0.5860

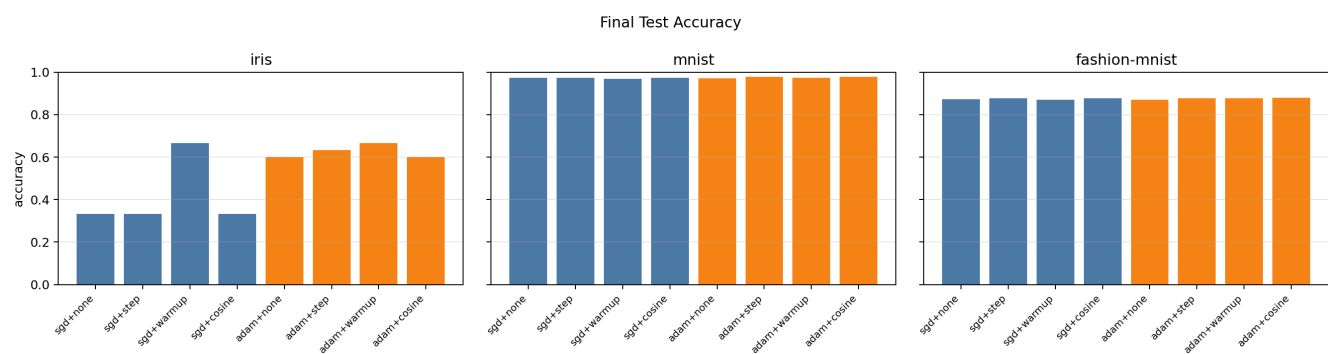
数据集	优化器	Scheduler	Final Test Acc	Best Test Acc	Best Epoch	Final Test Loss
Iris	SGD	cosine	0.3333	0.6000	6	7.4379
Iris	SGD	none	0.3333	0.6667	8	6.3045
Iris	SGD	step	0.3333	0.6667	9	3.7145
Iris	SGD	warmup	0.6667	0.7333	8	1.1612
MNIST	Adam	cosine	0.9789	0.9789	10	0.0764
MNIST	Adam	none	0.9710	0.9761	8	0.1128
MNIST	Adam	step	0.9776	0.9784	7	0.0875
MNIST	Adam	warmup	0.9732	0.9769	9	0.1066
MNIST	SGD	cosine	0.9736	0.9736	10	0.0856
MNIST	SGD	none	0.9723	0.9723	10	0.0918
MNIST	SGD	step	0.9741	0.9741	10	0.0849
MNIST	SGD	warmup	0.9680	0.9710	9	0.1047

汇总对比图如下。

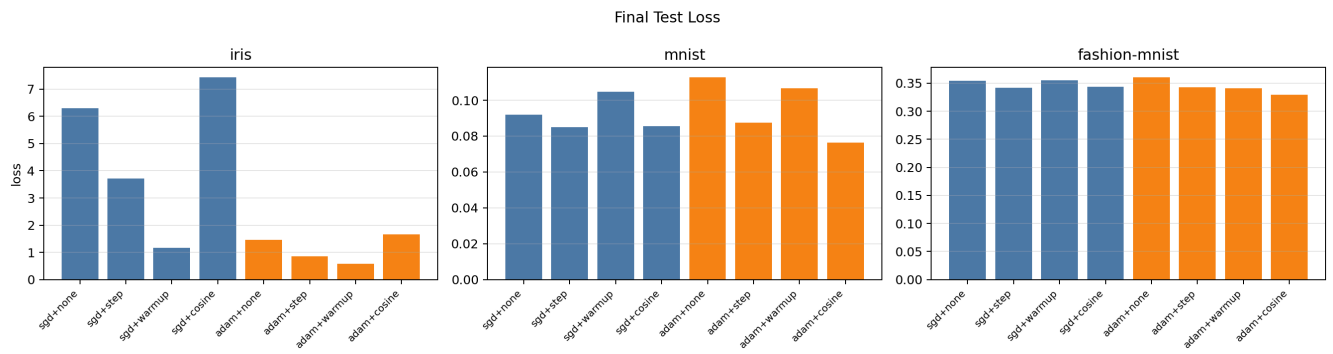
Best Test Accuracy 对比：



Final Test Accuracy 对比：

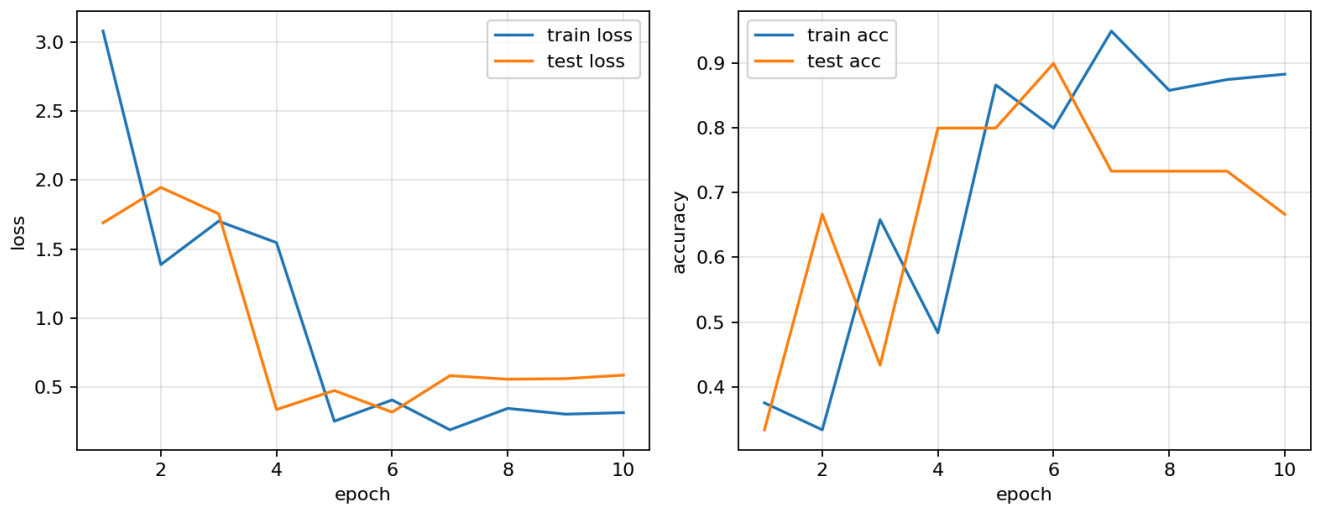


Final Test Loss 对比：

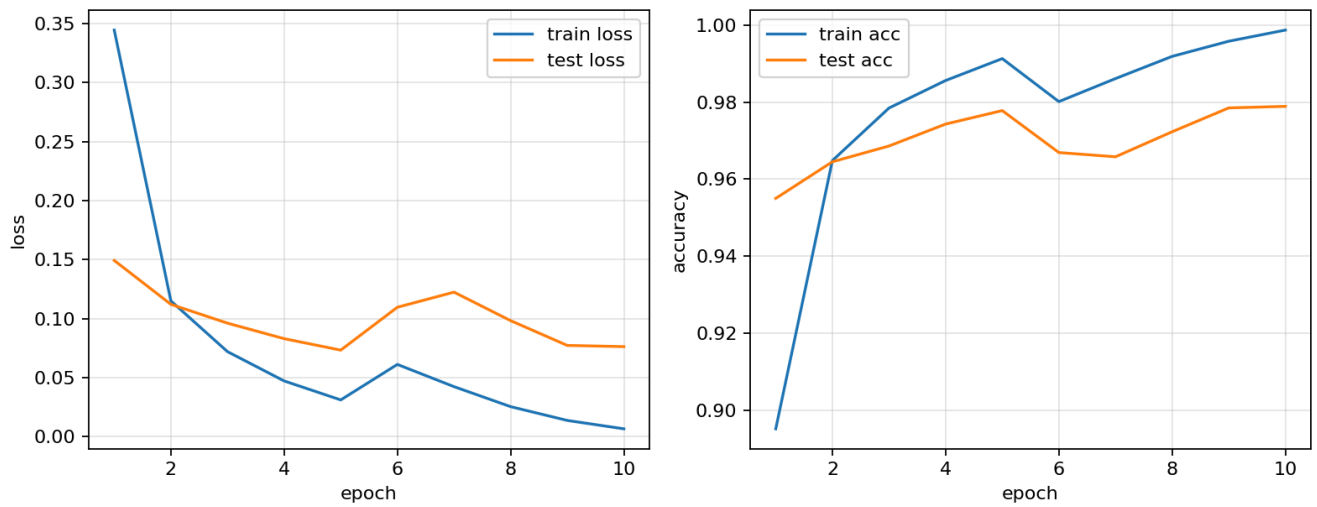


各数据集最佳实验的训练曲线如下。

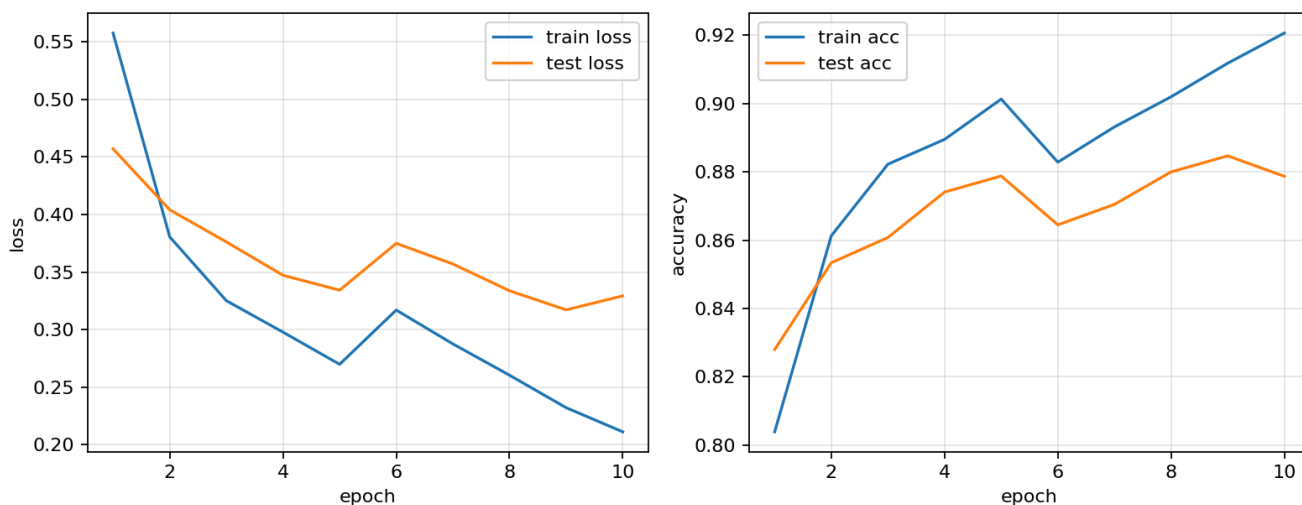
Iris 最佳配置为 Adam + warmup :



MNIST 最佳配置为 Adam + cosine :



Fashion-MNIST 最佳配置为 Adam + cosine :



5. 结果分析

从整体结果看，ResidualMLP 在 MNIST 上表现最好，最佳测试准确率达到 0.9789；在 Fashion-MNIST 上最佳测试准确率为 0.8847；在 Iris 上最佳测试准确率为 0.9000。

MNIST 的实验结果比较稳定。Adam 优化器整体略优于 SGD，其中 Adam + cosine 达到最高测试准确率 0.9789，并且最后一轮仍保持最佳效果。Adam + step 的最佳准确率也达到 0.9784，与 cosine 很接近。SGD 在 MNIST 上也表现良好，最佳配置为 SGD + step，测试准确率为 0.9741。这说明对于 MNIST 这种相对简单且样本量充足的数据集，ResidualMLP 已经能够稳定收敛，学习率调度可以带来小幅提升。

Fashion-MNIST 难度高于 MNIST，因此整体准确率较低。最佳配置为 Adam + cosine，最佳测试准确率为 0.8847。Adam 的四种 scheduler 都在 0.8800 左右，整体略优于 SGD。SGD 中表现最好的是 SGD + step，最终和最佳测试准确率均为 0.8779。这说明 Fashion-MNIST 中类别外观更复杂，仅使用展平图像输入的 MLP 无法像卷积网络那样利用局部空间结构，因此准确率提升有限。

Iris 的结果波动明显。最佳配置为 Adam + warmup，最佳测试准确率达到 0.9000，但最终测试准确率只有 0.6667。这是因为 Iris 测试集只有 30 个样本，每错一个样本准确率就会变化约 3.33%，因此结果对单个样本非常敏感。SGD 的多个配置最终准确率只有 0.3333，接近三分类随机猜测，说明在本实验设置下 SGD 对 Iris 的训练不稳定。相比之下，Adam 更容易找到较好的参数区域，尤其 warmup 能缓解训练初期更新过猛的问题。

从优化器角度看，Adam 在三个数据集上整体更稳。在 MNIST 和 Fashion-MNIST 上，Adam 的最佳结果均由 cosine scheduler 获得；在 Iris 上，Adam + warmup 最好。SGD 在 MNIST 和 Fashion-MNIST 上可以训练到较高准确率，但在 Iris 上表现不稳定。

从 scheduler 角度看，cosine 在 MNIST 和 Fashion-MNIST 上最有优势，step 也能稳定提升或接近最优。warmup 对 Iris 帮助最大，但在 MNIST 和 Fashion-MNIST 上不一定优于 cosine 或 step。none 作为固定学习率基线，在 MNIST 和 Fashion-MNIST 上也能得到不错结果，但一般不是最优。

综合来看，本次系统实验中推荐的配置为：

数据集	推荐配置	Best Test Acc
Iris	Adam + warmup	0.9000
MNIST	Adam + cosine	0.9789
Fashion-MNIST	Adam + cosine	0.8847

这组结果说明当前实现的自动微分、基础算子、神经网络模块、优化器、scheduler、DataLoader 和训练脚本已经能够支持较完整的神经网络分类实验。